

GRAPH

PROBLEM 10

QUESTION:

DS	Session	Graph	Question Information	Level 1	Challenge 90
Question description Kaaleppi has just robbed a bank and is now heading to the harbor. However, the police wants to stop him by closing some streets of the city. What is the minimum number of streets that should be closed so that there is no route between the bank and the harbor? Constraints • $2 \leq n \leq 500$ • $1 \leq m \leq 1000$ • $1 \leq a, b \leq n$ Input The first input line has two integers n and m: the number of crossings and streets. The crossings are numbered $1, 2, \dots, n$. The bank is located at crossing 1, and the harbor is located at crossing n. After this, there are m lines that describe the streets. Each line has two integers a and b: there is a street between crossings a and b. All streets are two-way streets, and there is at most one street between two crossings. Output First print an integer k: the minimum number of streets that should be closed. After this, print k lines describing the streets. You can print any valid solution.					

Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) <pre>4 5 1 2 1 3 2 3 3 4 1 4</pre> EXPECTED OUTPUT <pre>2 1 4 3 4</pre>	INPUT (STDIN) <pre>4 5 1 1 2 2 2 3 3 4 1 4</pre> EXPECTED OUTPUT <pre>1 1 4</pre>

Mandatory Test Cases

Test Case 1	Test Case 2
KEYWORD <pre>void link(int i,int h)</pre>	KEYWORD <pre>int bfs(int n,int s,int t)</pre>

Complexity Test Cases

Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY <pre>1</pre>	TOKEN COUNT <pre>735</pre>	NLOC <pre>93</pre>

SOURCE CODE:

```

#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAX_STATIONS 505
#define MAX_STREETS 2005

typedef struct DirectedStreet {
    int target, cap, flow, rev;
} DirectedStreet;

DirectedStreet streets[4 * MAX_STREETS];
int station_head[MAX_STATIONS], next_street[4 * MAX_STREETS], street_cnt = 0;

int input_src[MAX_STREETS], input_dst[MAX_STREETS];

void add_bidirectional_flow(int u, int v, int capacity) {
    streets[street_cnt] = (DirectedStreet){v, capacity, 0, street_cnt + 1};
    next_street[street_cnt] = station_head[u];
    station_head[u] = street_cnt++;

    streets[street_cnt] = (DirectedStreet){u, capacity, 0, street_cnt - 1};
    next_street[street_cnt] = station_head[v];
    station_head[v] = street_cnt++;
}

int prev_street[MAX_STATIONS];

bool find_chase_path(int n, int src, int dst) {
    for (int i = 1; i <= n; i++) prev_street[i] = -1;
    int queue[MAX_STATIONS], head = 0, tail = 0;
    queue[tail++] = src;
    prev_street[src] = -2;

    while (head < tail) {
        int cur = queue[head++];
        if (cur == dst) return true;
        for (int s = station_head[cur]; s != -1; s = next_street[s]) {
            int nxt = streets[s].target;
            if (prev_street[nxt] == -1 && streets[s].cap > streets[s].flow) {
                prev_street[nxt] = s;
                queue[tail++] = nxt;
            }
        }
    }
    return false;
}

int main() {
    int stations, num_streets;
    if (scanf("%d %d", &stations, &num_streets) != 2) return 0;

    for (int i = 1; i <= stations; i++) station_head[i] = -1;
    for (int i = 0; i < num_streets; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        input_src[i] = u;
        input_dst[i] = v;
        add_bidirectional_flow(u, v, 1);
    }

    int max_flow_val = 0;
    while (find_chase_path(stations, 1, stations)) {
        for (int v = stations; v != 1; ) {
            int s = prev_street[v];
            streets[s].flow += 1;
            streets[streets[s].rev].flow -= 1;
            v = streets[streets[s].rev].target;
        }
        max_flow_val++;
    }

    bool reachable_from_bank[MAX_STATIONS] = {false};
    int bfs_q[MAX_STATIONS], q_h = 0, q_t = 0;
    bfs_q[q_t++] = 1;
    reachable_from_bank[1] = true;
    while (q_h < q_t) {
        int cur = bfs_q[q_h++];
        for (int s = station_head[cur]; s != -1; s = next_street[s]) {
            int nxt = streets[s].target;
            if (!reachable_from_bank[nxt] && streets[s].cap > streets[s].flow) {
                reachable_from_bank[nxt] = true;
                bfs_q[q_t++] = nxt;
            }
        }
    }

    printf("%d\n", max_flow_val);
    for (int i = 0; i < num_streets; i++) {
        int u = input_src[i];
        int v = input_dst[i];
        if ((reachable_from_bank[u] && !reachable_from_bank[v]) ||
            (reachable_from_bank[v] && !reachable_from_bank[u])) {
            printf("%d %d\n", u, v);
        }
    }
    return 0;
}

```

OUTPUT:

```
4 5
1 2
1 3
2 3
2 4
3 4
2
2 4
3 4
```

```
4 5
1 1
2 2
2 3
3 4
1 4
1
1 4
```